

CS4227_CS5722 - Software Architecture

Designing the architecture and Evaluating a Component-Based Research Management System

Semester 2 AY 24/25

Assignment 2

Module: Software Architecture

Rohan Sikder - 24165816

Date:

11/03/2025

Table of Contents

Summary ***3***

Architectural Style & Justification ***3***

Event-Driven Architecture (EDA) **3**

Why Event-Driven Architecture? 3

Key Components of EDA in RMS: 3

UML Diagrams ***4***

1.1 Use Case Diagram 4

1.2 Sequence Diagram - Creating a Thread 4

1.3 Deployment Diagram 4

1.4 Component Diagram 4

Frontend Design ***7***

Key Components: **7**

GitHub Repository: **9**

Github Repo: 9

Github Pages: 9

UX/UI Design: **9**

Backend Implementation ***9***

Event-Driven Architecture (EDA): 9

State Management: **10**

Code Structure: **10**

Conclusion ***11***

Completed Work: **11**

Future Work: **11**

Bibliography ***11***

Summary

The research Management System (RMS) is a System for collaborating, discussing and organizing Research environments. It enables users to create and participate in discussion threads, tag other users and get real time notifications of corresponding behaviors. It is a scalable, modular and real-time responsive Architecture based on Event-Driven architecture (EDA). The frontend is made to React using TypeScript and the backend logic is implemented within the frontend using TypeScript for quickly prototyping.

Architectural Style & Justification

Event-Driven Architecture (EDA)

Event-Driven Architecture (EDA) is adopted for RMS due to its ability to process real-time notifications and user interactions efficiently. In EDA, components communicate via events, triggered by user actions or system processes. This architecture is especially suited for real-time updates (new thread, mention, reply etc.).

Why Event-Driven Architecture?

- Real-Time notifications: EDA allows the system to send real-time Notifications when new threads are created, replies are posted or users are mentioned in discussions.

- Scalability: EDA is scalable because of the asynchronous processing of events which enable the system to process many users and interactions with minimal performance degradation.

- Modularity: The system is organized into decoupled components (discussion threads, notifications, user mentions) that can be independently developed and scaled.

- Interoperability: EDA provides interoperability with various systems and services including external notification systems or third-party APIs.

Key Components of EDA in RMS:

- Event Producers: Elements that initiate events (creating a new thread, posting a reply or tagging a user).

- Event Consumers: Reacting Components that send notifications or update the UI in response to an event.

- Event Bus: A centralized event communication system that connects event producers to consumers. In this implementation, the event bus is handled by mitt, a simple event emitter library.

UML Diagrams

The UML diagrams provide a representation of the Research Management System (RMS). They outline the interactions, components and deployment of the system.

1.1 Use Case Diagram

- The Use Case Diagram presents the functionalities available to users in RMS.
- Key actions include:
    - Creating Discussions
    - Replying to Threads
    - Mentioning Users
    - Receiving Real-time Notifications
    - Filtering and Sorting Discussions
    - Managing Preferences (e.g., Theme Toggle)

1.2 Sequence Diagram - Creating a Thread

- This diagram visualizes how users create new discussion threads and receive notifications.
- It follows an event-driven model:
    - A user creates a thread in the frontend UI (Discussions.tsx).
    - The eventBus (implemented in eventBus.ts) emits a newThread event.
    - The Notification System (Notifications.tsx and NotificationSystem.tsx) listens for this event and updates the UI.

Since this is a frontend-only prototype, there is no external backend API. The logic is handled entirely within React components using event-based communication**.**

1.3 Deployment Diagram

- The system is structured as:
    - Client-Side (React Frontend)
    - Event-Driven Logic (Event Bus)
    - Browser Storage (LocalState Management)
- The absence of a backend means that all data persistence is limited to state management within the React app.

1.4 Component Diagram

- The Component Diagram maps the modular structure of the frontend:
    - **App.tsx**: Manages routing (react-router-dom) and the global theme.
    - **Discussions.tsx**: Handles discussion threads, mentions, and replies.
    - **Notifications.tsx**: Displays user notifications.
    - **eventBus.ts**: Implements the event-driven communication model for handling real-time updates without a backend.

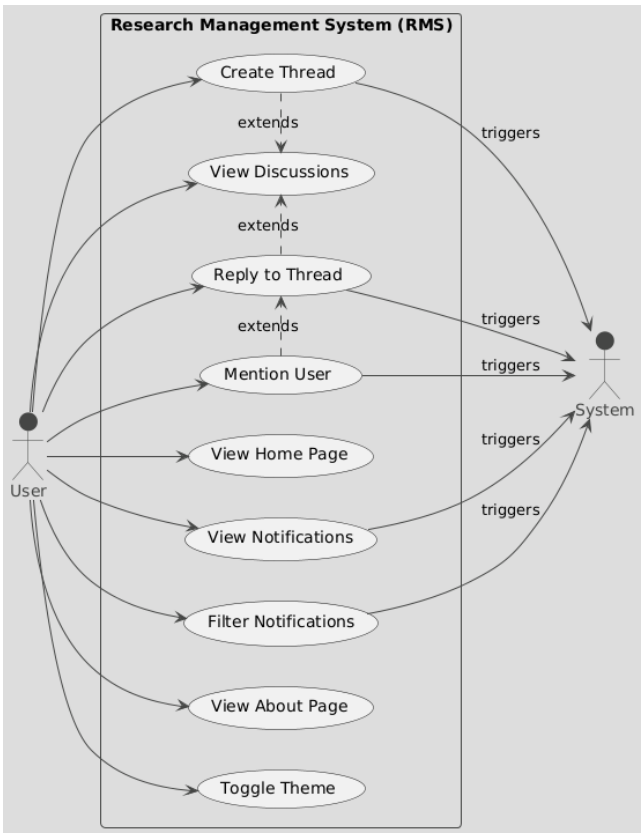


Fig 1. Use Case Diagram for Research Management System.

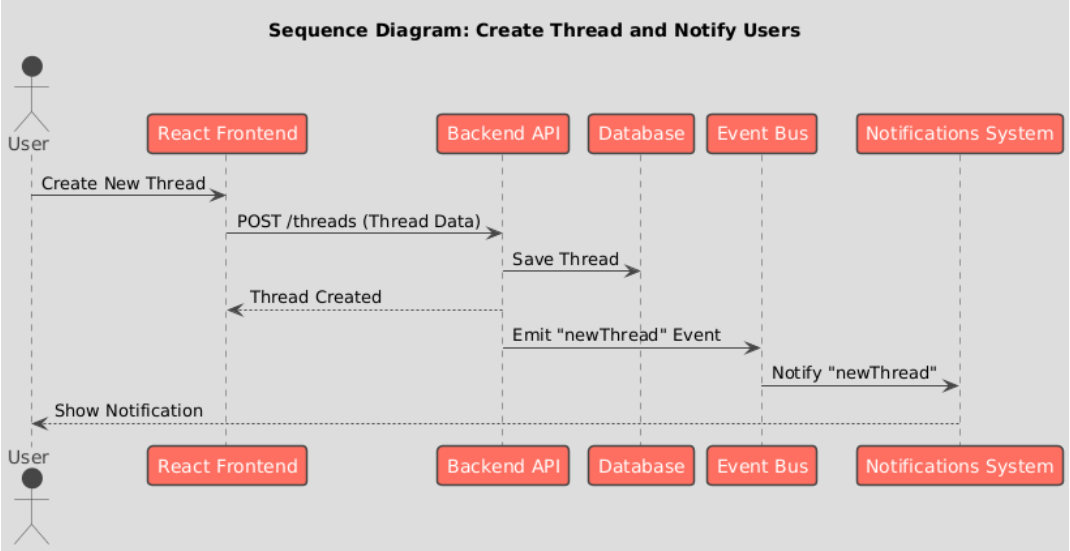


Fig 2. Sequence Diagram for creating a thread and notifying users.

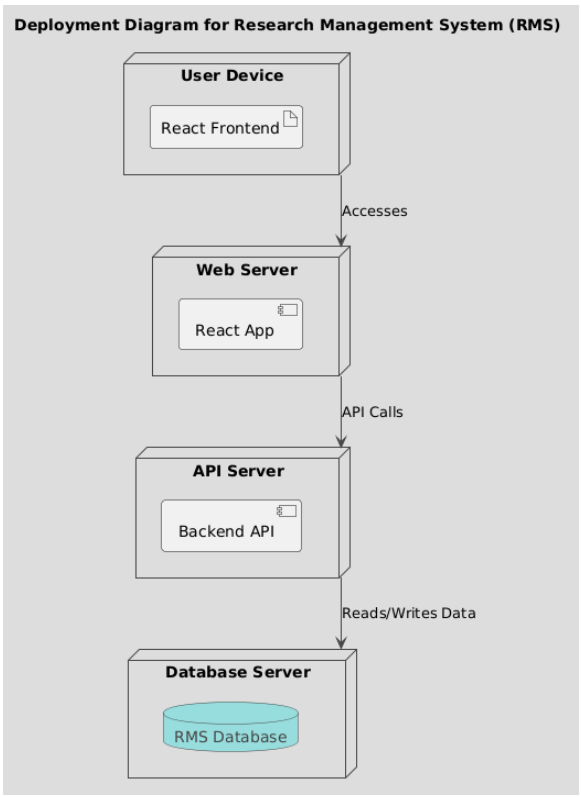


Fig 3. Proof of Concept Deployment Diagram

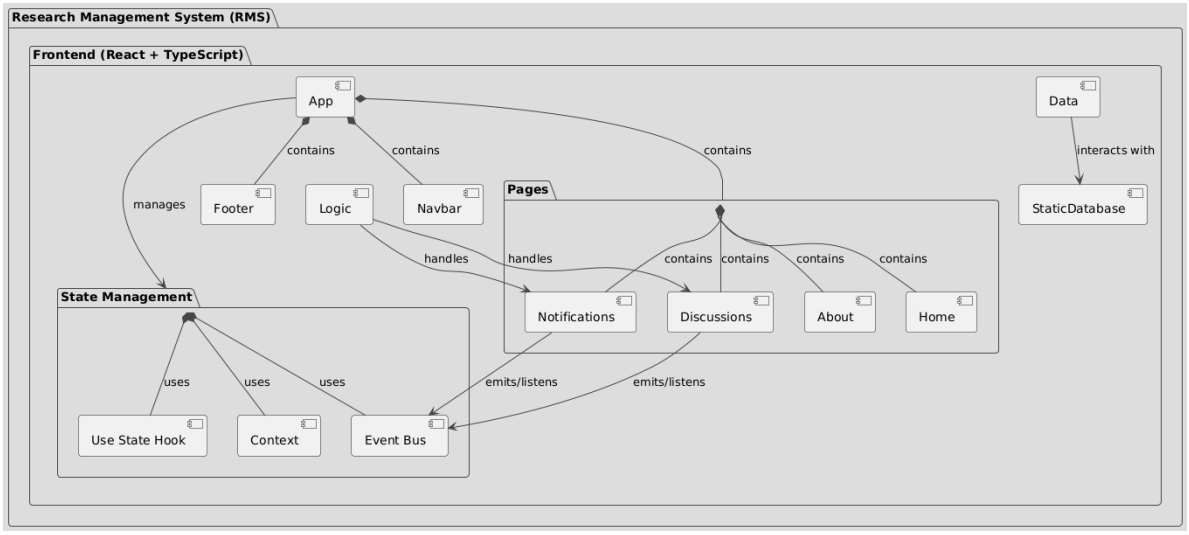


Fig 4. Component Diagram

Frontend Design

The Research Management System (RMS) frontend is modular and scalable. It is developed in React with TypeScript for type safety and easier maintenance of the codebase. The frontend is split into many main parts which serve different purposes:

Key Components:

1. App.tsx:

- This is the main application entry point. It handles routing via react-router-dom and handles the global theme (light / dark mode). The Navbar element is rendered here, which contains navigation links to various pages (Home, Discussions, Notifications, About).

2. Discussions.tsx:

- This component allows users to create and participate in discussion threads. It supports features like:
    - Creating new threads with titles, content, and subjects.
    - Replying to threads and tagging users using the @mention functionality.
    - Upvoting and downvoting threads and replies.
    - Subscribing to threads to receive notifications for new replies.
    - Sorting and filtering threads by subjects, popularity, or subscription status.

3. Notifications.tsx:

- This component handles real-time notifications for users. It listens for events such as:
    - New thread creation.
    - User mentions in replies.
    - New replies in subscribed threads.
- Notifications are displayed in a card format and can be filtered by type (new thread, mention, subscribed reply).

4. eventBus.ts:

- This is the main event bus for interconnectivity between components. It calls the mitt library and emits and listens for newThread, mention and subscribedReply events. This event bus ensures that the system can handle real-time updates without requiring a backend server.

5. About.tsx:

- This component provides information about the RMS, including its key features and benefits. It is a static page that helps users understand the purpose and functionality of the system.

6. Home.tsx:

- The home page displays user stories and acceptance criteria for the RMS. It serves as a guide for users to understand the system's capabilities and how it can be used to facilitate research collaboration.

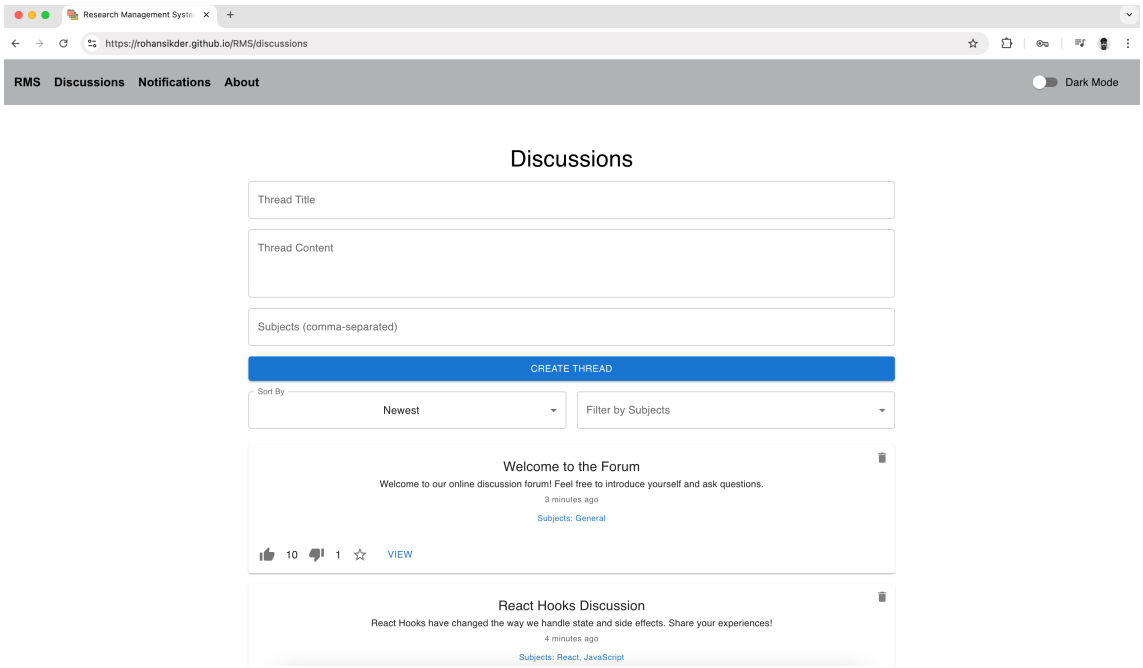


Fig 5. Discussions.tsx component allows users to create and participate in discussion threads.

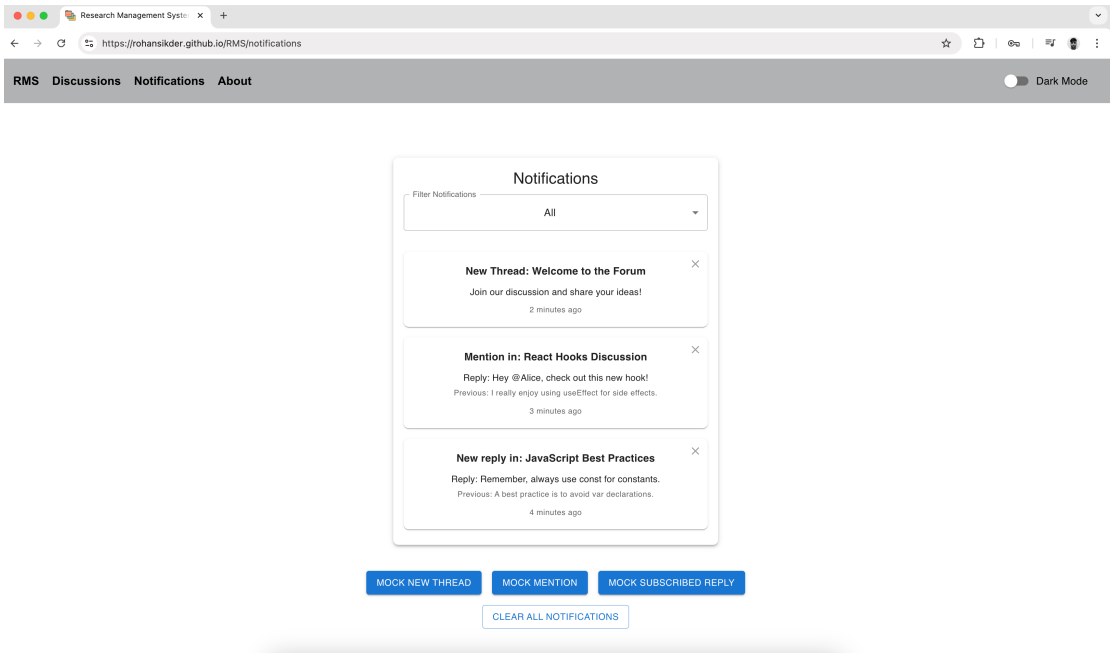


Fig 6. Notifications.tsx Component handles real-time notifications for users.

GitHub Repository:

You can find the complete source code for the RMS prototype on GitHub:

Github Repo: https://github.com/rohansikder/RMS

Github Pages: https://rohansikder.github.io/RMS

UX/UI Design:

The RMS frontend was developed using Material-UI (MUI) (version 4) [MUI], a popular React UI framework built with Google's Material Design guidelines. MUI offered a library of preconfigured, configurable elements to build a simple, responsive and intuitive interface. The application was modeled using Cards, Buttons, TextFields and Dialogs using key components and light/dark mode toggling was implemented using MUI's theming system. The MUI kept application consistency consistent across the application and simplified development by removing the need for custom CSS.

Backend Implementation

As the RMS is a frontend-only prototype, the backend logic is embedded in the frontend through TypeScript. This approach enables rapid development and testing of the system's core functionalities without the requirement for a separate backend server. Below is an overview of how the backend logic is structured within the frontend:

Event-Driven Architecture (EDA):

The RMS uses an Event-Driven Architecture (EDA) to handle real-time updates and user interactions. The key components of this architecture are:

1. Event Producers:

- These are the components that initiate events, such as:
    - Creating a new thread (newThread event).
    - Posting a reply (mention event if a user is tagged).
    - Subscribing to a thread (subscribedReply event).

2. Event Consumers:

- These components listen for events and update the UI accordingly. For example:
    - The Notifications.tsx component listens for newThread, mention, and subscribedReply events to display real-time notifications.
    - The Discussions.tsx component listens for events to update the discussion threads and replies in real-time.

3. Event Bus:

- The eventBus.ts file acts as the central hub for event communication. It uses the mitt library to emit and listen for events. This ensures that all components can communicate with each other in a decoupled manner.

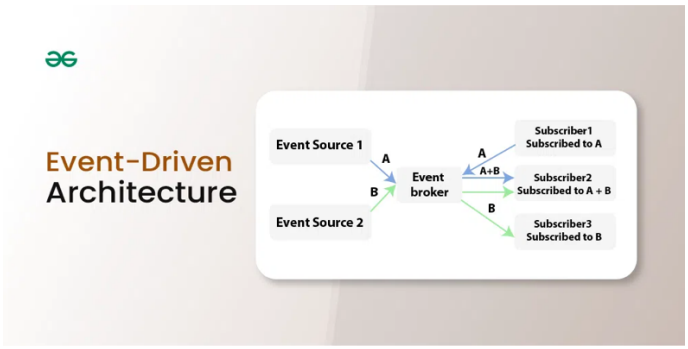


Fig 7. Event-Driven Architecture Diagram (GeekForGeeks.com)

State Management:

The frontend is handled by React's state management, while the RMS handles data persistence. Because there is no backend, all data (threads, replies, notifications) is cached in the browser's memory. This approach is sufficient for prototyping but would need to be replaced by a proper backend database in production.

Code Structure:

The project is organized into the following structure:

- **src/pages/**: Contains the main pages of the application (Home, Discussions, Notifications, About).
- **src/utils/**: Contains eventBus.ts for event handling.
- **src/App.tsx**: The main entry point of the application.

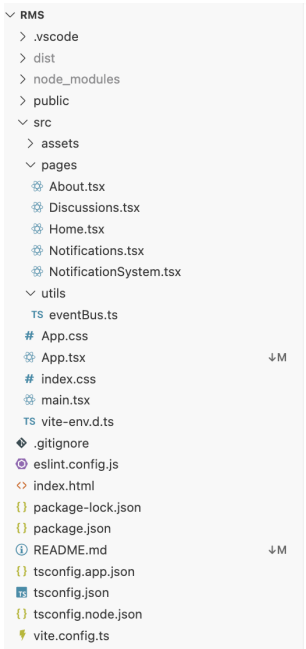


Fig 8. Code Structure

Conclusion

Completed Work:

- **Frontend Development**: The frontend of the RMS has been successfully implemented using React and TypeScript. Key features such as discussion threads, real-time notifications and user mentions are fully functional.
- **Event-Driven Architecture**: The system uses an event-driven architecture to handle real-time updates and user interactions. The eventBus.ts file facilitates communication between components, ensuring a modular and scalable design.
- **Prototype Testing**: The prototype has been tested for basic functionalities, including creating threads, replying to threads and receiving real-time notifications.

Future Work:

- **Backend Integration**: In the next phase, the system will be integrated with a backend server to handle data persistence, user authentication which other team members have implemented where user log in was created, and more complex business logic.
- **Database Implementation**: A database (e.g., MongoDB) will be added to store threads and notifications.
- **User Authentication**: User roles and authentication will be implemented to restrict access to certain features based on user permissions which would be tied in with other team members components where user based roles was created..
- **Advanced Features**: Additional features such as file uploads and advanced search will be added to enhance the system's functionality.
- **Component Integration:** Integration of all components from all team members of RMS to create application which has all features interconnected.

Bibliography

1. [GeeksForGeeks]: https://www.geeksforgeeks.org/event-driven-architecture-system-design/
2. [Mitt]: https://www.npmjs.com/package/mitt/v/1.0.1
3. [React]: https://react.dev/
4. [MUI]: https://mui.com/
5. [PlantUML]: https://www.plantuml.com/
6. [Typescript]: https://www.typescriptlang.org/